TIMER1

Equation

$$\text{Required delay} = \frac{4*\text{prescalar}*(2^{16}-1)-\text{TMR1}}{\text{FOSC}}$$

FOSC=20MHz

TIMER2

Equation

$$\text{Required delay} = \frac{4*\text{prescalar}*\text{postscalar}*\text{PR2}}{\text{FOSC}}$$

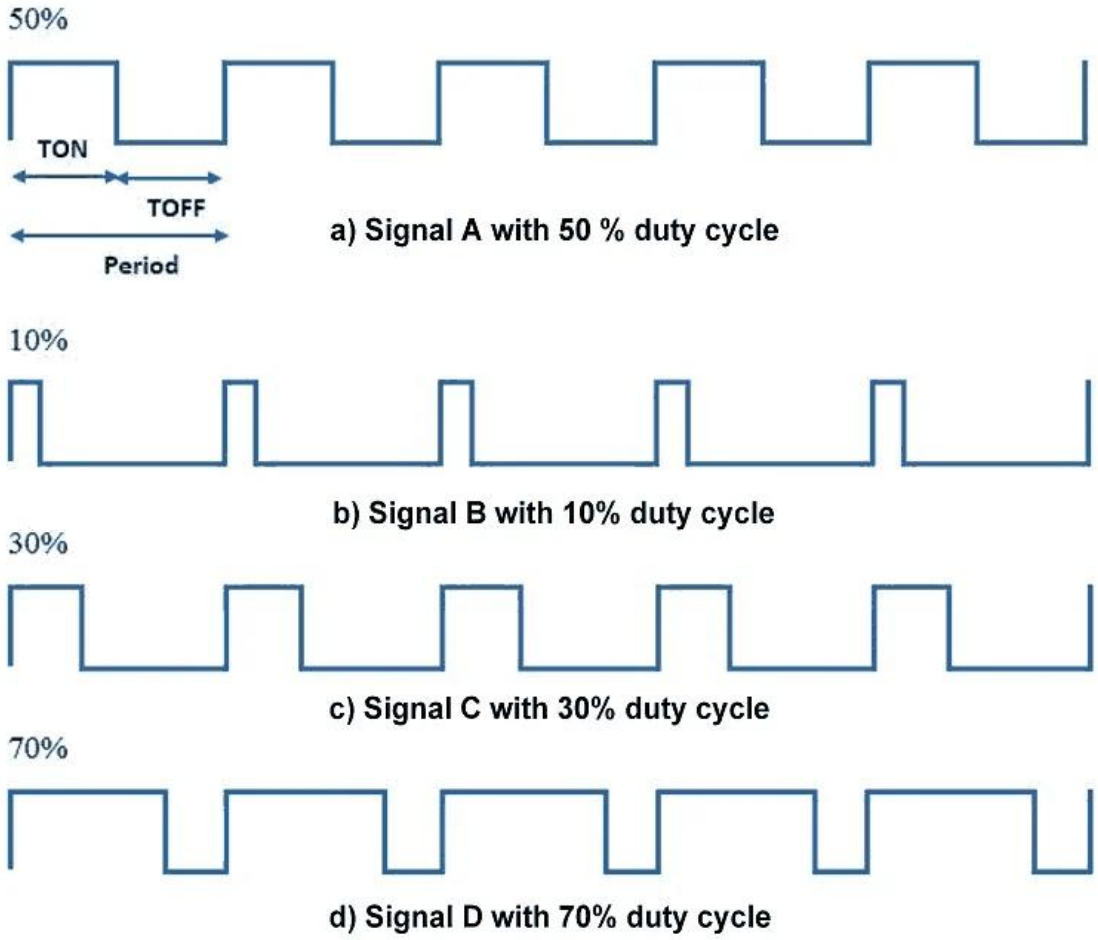
FOSC=20MHz

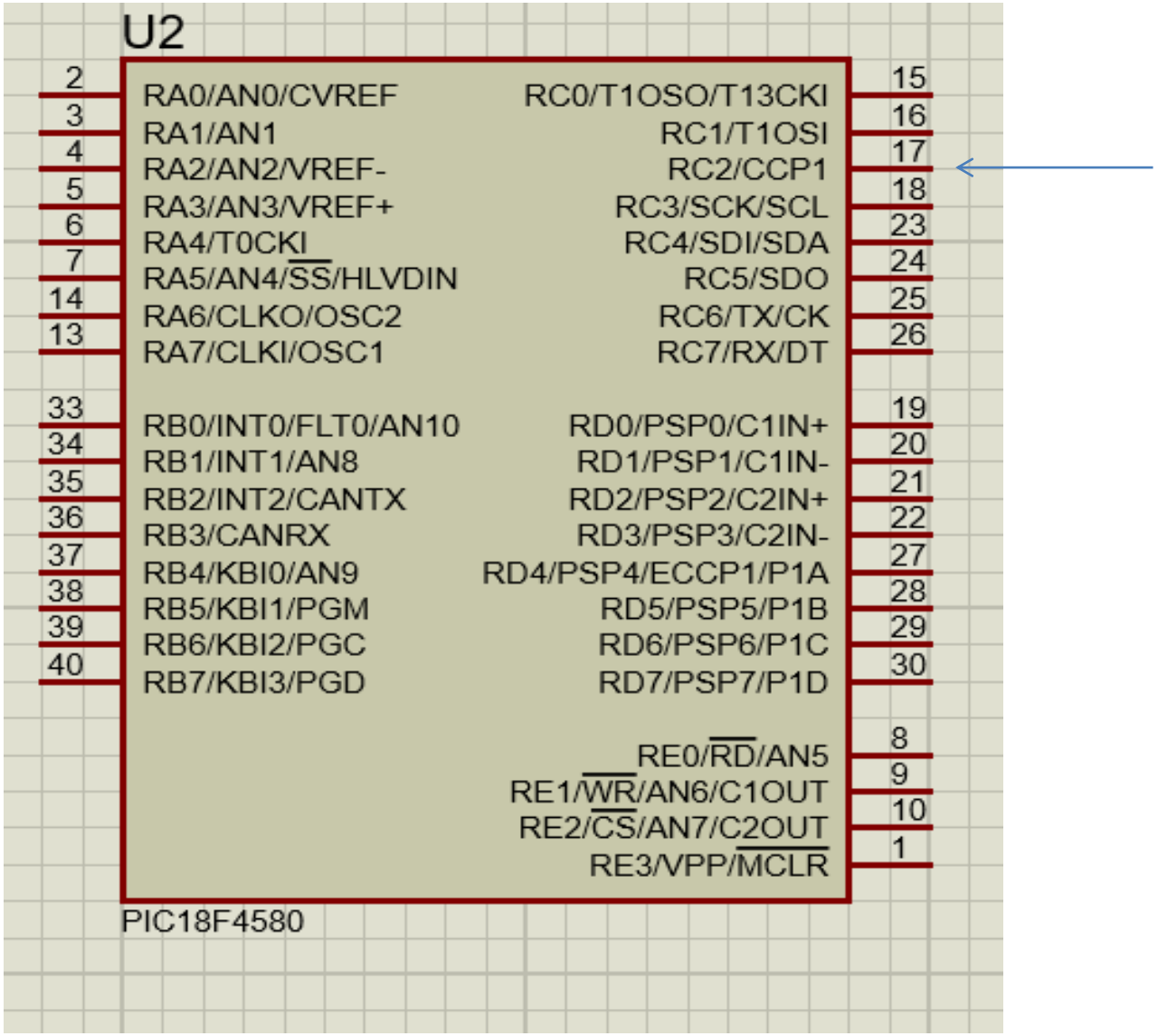
PWM

Pulse Width Modulation (PWM) is a technique by which the width of a pulse is varied while keeping the frequency of the wave constant.

A period of a pulse consists of an **ON** cycle (5V) and an **OFF** cycle (0V). The fraction for which the signal is ON over a period is known as a **duty cycle**.

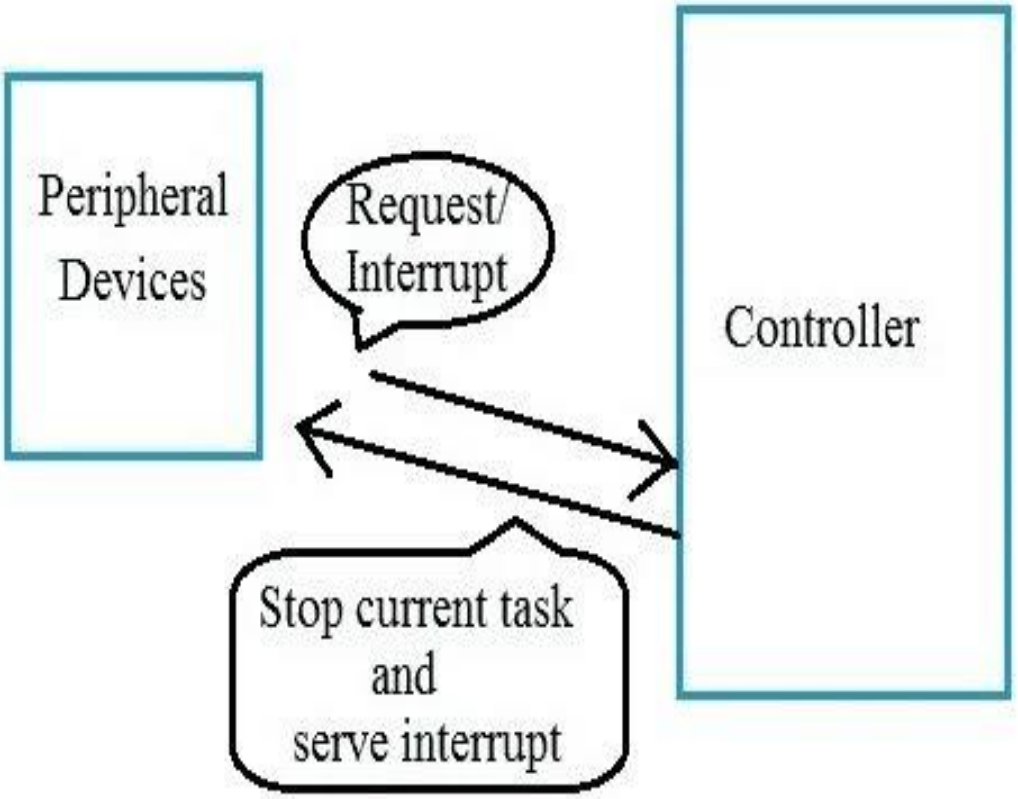
$$DutyCycle(percentage) = \frac{Ton}{TotalPeriod} * 100$$


50%
TON
TOFF
Period
a) Signal A with 50 % duty cycle
10%
b) Signal B with 10% duty cycle
30%
c) Signal C with 30% duty cycle
70%
d) Signal D with 70% duty cycle

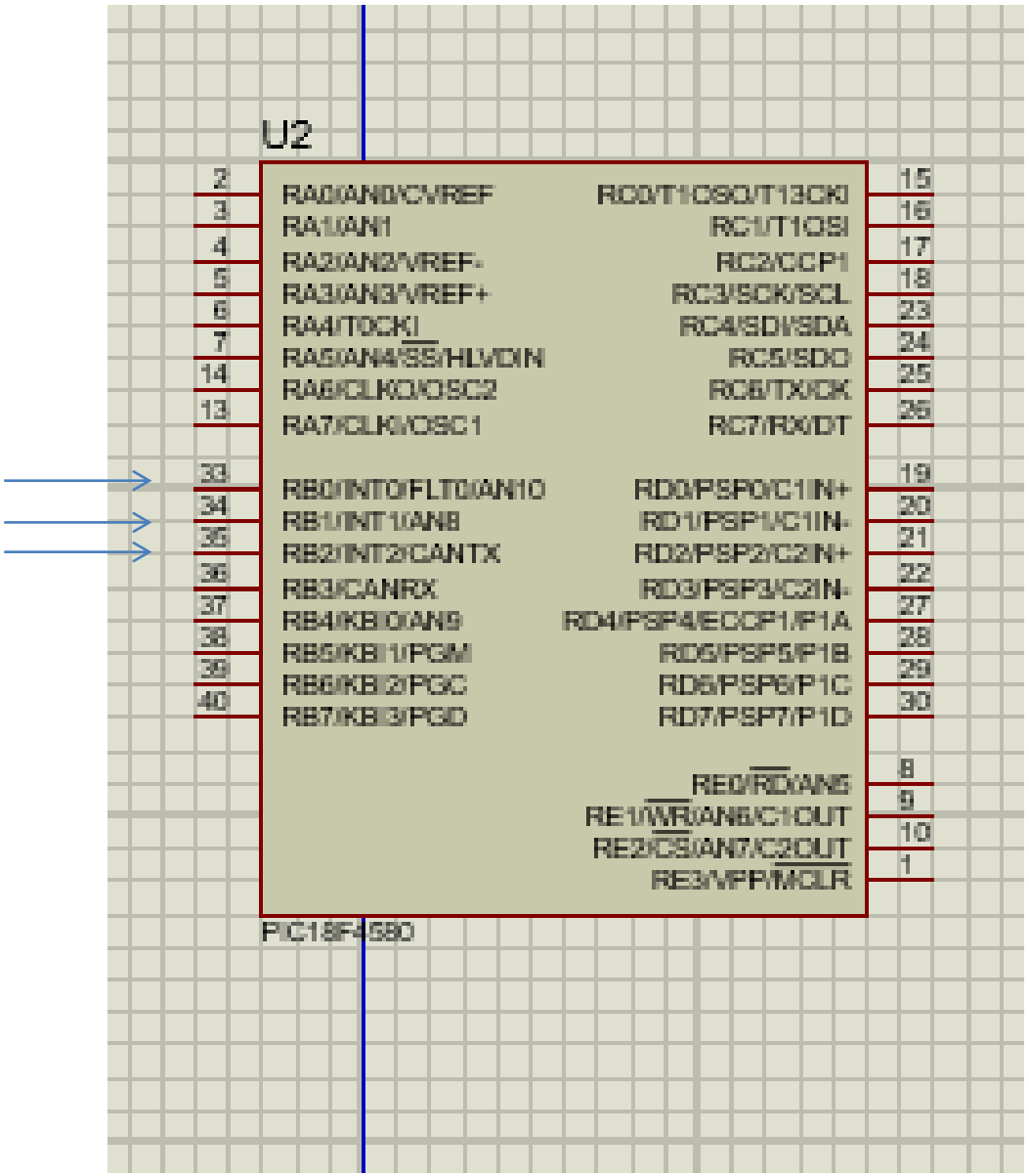

U2
2 RA0/AN0/CVREF
3 RA1/AN1
4 RA2/AN2/VREF-
5 RA3/AN3/VREF+
6 RA4/T0CKI
7 RA5/AN4/SS/HLVDIN
14 RA6/CLKO/OSC2
13 RA7/CLKI/OSC1
33 RB0/INT0/FLT0/AN10
34 RB1/INT1/AN8
35 RB2/INT2/CANTX
36 RB3/CANRX
37 RB4/KBI0/AN9
38 RB5/KBI1/PGM
39 RB6/KBI2/PGC
40 RB7/KBI3/PGD
15 RC0/T1OSO/T13CKI
16 RC1/T1OSI
17 RC2/CCP1
18 RC3/SCK/SCL
23 RC4/SDI/SDA
24 RC5/SDO
25 RC6/TX/CK
26 RC7/RX/DT
19 RD0/PSP0/C1IN+
20 RD1/PSP1/C1IN-
21 RD2/PSP2/C2IN+
22 RD3/PSP3/C2IN-
27 RD4/PSP4/ECCP1/P1A
28 RD5/PSP5/P1B
29 RD6/PSP6/P1C
30 RD7/PSP7/P1D
8 RE0/RD/AN5
9 RE1/WR/AN6/C1OUT
10 RE2/CS/AN7/C2OUT
1 RE3/VPP/MCLR
PIC18F4580

Interrupt

Interrupts are the signals which alter the flow of an executing program by causing the microcontroller jumps to the Interrupt Service Routine to serve to interrupt. These interrupts are given to the microcontroller unit through the external pins of the microcontroller.


Peripheral Devices
Request/ Interrupt
Controller
Stop current task and serve interrupt

- Once the controller completes the routine, it returns to the location from where it had made a jump.
- In cases where interrupts are not used, the program would need to constantly poll the input signals to monitor external events for catching the pulses when they occurred. But in some cases, the program can miss an event.
- E.g. An infrared slot sensor trying to catch a coin drop. In this situation, using an interrupt can free the microcontroller to get some other work done while not missing the input. In such cases, external interrupts are used.


U2
2 RA0/AN0/CVREF
3 RA1/AN1
4 RA2/AN2/VREF-
5 RA3/AN3/VREF+
6 RA4/T0CKI
7 RA5/AN4/SS/HLVDIN
14 RA6/CLKO/OSC2
13 RA7/CLKI/OSC1
33 RB0/INT0/FLT0/AN10
34 RB1/INT1/AN8
35 RB2/INT2/CANTX
36 RB3/CANRX
37 RB4/KBI0/AN9
38 RB5/KBI1/PGM
39 RB6/KBI2/PGC
40 RB7/KBI3/PGD
15 RC0/T1OSO/T13CKI
16 RC1/T1OSI
17 RC2/CCP1
18 RC3/SCK/SCL
23 RC4/SDI/SDA
24 RC5/SDO
25 RC6/TX/CK
26 RC7/RX/DT
19 RD0/PSP0/C1IN+
20 RD1/PSP1/C1IN-
21 RD2/PSP2/C2IN+
22 RD3/PSP3/C2IN-
27 RD4/PSP4/ECCP1/P1A
28 RD5/PSP5/P1B
29 RD6/PSP6/P1C
30 RD7/PSP7/P1D
8 RE0/RD/AN5
9 RE1/WR/AN6/C1OUT
10 RE2/CS/AN7/C2OUT
1 RE3/VPP/MCLR
PIC18F4580

High priority and Low priority

In embedded systems, high-priority interrupts (HPI) are reserved for time-critical, emergency tasks (e.g., motor faults, safety sensors) that must run immediately, even interrupting lower-priority ISRs. Low-priority interrupts (LPI) handle routine tasks (e.g., UI updates, serial data) and are paused if an HPI occurs, only executing once higher-priority tasks finish.

Maskable and Unmaskable interrupt

Maskable Interrupts **(MI)** are signals that the CPU can ignore or temporarily disable via software (e.g., peripheral I/O), generally for lower-priority tasks. **Non-Maskable Interrupts** **(NMI)** cannot be ignored or disabled